



Object Oriented Programming – CS/CE 224/272

Nadia Nasir





Character Arrays / C-Style Strings

C-Style Strings



Inherited from C programming language into C++.

C-style strings are **character arrays**.

They are **null-terminated** strings.

C-style strings have a **null character** (`'\0'`) at their end.

This character has an ASCII code = 0.

`std::string` in C++ is also based on C-style strings.



LHS
Operand
ostream
type object

operator<<

RHS Operand
dtype of data
being printed

*Elaboration on the
different behavior
with std::cout*

std::cout << (int) // Prints the int

std::cout << (int*) // Prints the address

std::cout << (double*) // Prints the address

std::cout << (char*) // Prints the data

*Somewhat unique
behavior.*

*Recall this
diagram...*

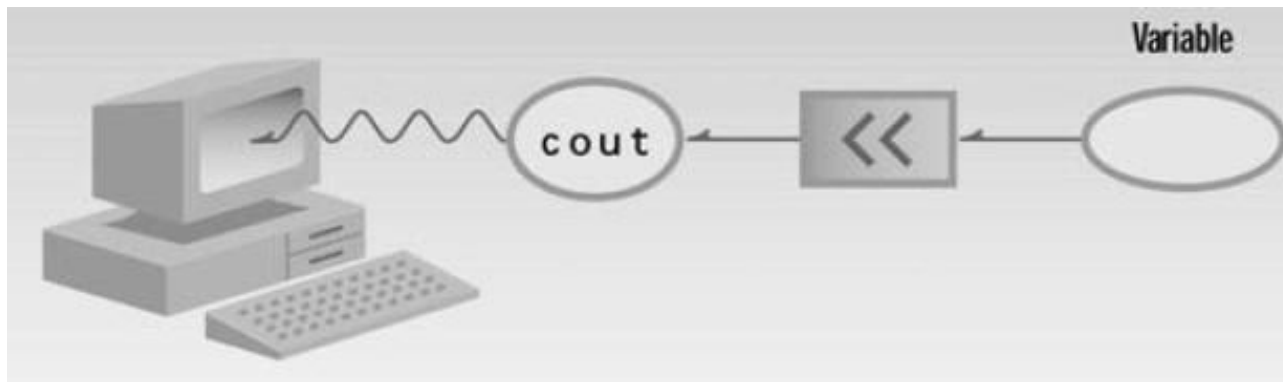


Figure 2.2 from OOP in C++ by Robert Lafore.



Void Pointers (`void*`)

So how to print a **`char*`** type address if we want to?

The **void pointer**, is a special type of pointer that can point at objects of **any data type**.

It could also point to objects (whose data types are created by yourself).



Void Pointers (`void*`) – Limitations

A void pointer has **no type information**.

Consequently, the following things happen.

You cannot dereference it.

You'll have to cast it to another pointer type first.

You cannot do pointer arithmetic with it.

Void Pointers (void*) – Limitations



```
int num{ 5 };  
void* v_ptr{ &num };  
  
std::cout << "Value of num = " << *( static_cast<int*>(v_ptr) ) << std::endl;
```

`#include<cstring>`

Provides helper functions for use with C-style strings



C-Style Strings – Helper Functions

There are functions in the **<cstring>** header file to work with C-style strings.

```
#include<cstring>
```

std::strcpy()



```
1  #include <cstring>
2  #include <iostream>
3
4  int main()
5  {
6      char source[] { "Copy this!" };
7      char dest[50];
8      std::strcpy(dest, source);
9      std::cout << dest << '\n'; // prints "Copy this!"
10
11     return 0;
12 }
```

std::strcpy()

Ensure, that the destination array can store all the elements in the source.

```
1 | #include <cstring>
2 | #include <iostream>
3 |
4 | int main()
5 | {
6 |     char source[]{ "Copy this!" };
7 |     char dest[5]; // note that the length of dest is only 5 chars!
8 |     std::strcpy(dest, source); // overflow!
9 |     std::cout << dest << '\n';
10 |
11 |     return 0;
12 | }
```

std::strlen()



Gives you the length of the string without the null character.

```
1  #include <iostream>
2  #include <cstring>
3
4  int main()
5  {
6      char name[20]{ "Alex" }; // only use 5 characters (4 letters + null terminator)
7      std::cout << "My name is: " << name << '\n';
8      std::cout << name << " has " << std::strlen(name) << " letters.\n";
9      std::cout << name << " has " << sizeof(name) / sizeof(name[0]) << " characters in the array.\n";
10
11     return 0;
12 }
```


Other Functions



Appends one string to another.

strcat()

Compares two strings.

strcmp()

More functions present in the provided reference.



General Recommendation

Avoid using C-style strings if you can.

Have various dangers associated with them.

Prefer **std::string** over C-style strings.

#include<string>

For the purposes of this course, however, you will be expected to use/know **C-style strings** in various assessments.

They are used in existing codes and various use cases.

So you should understand how they work.

References & *Pointers*



Pointers

Pointers

- `int var = 4;`

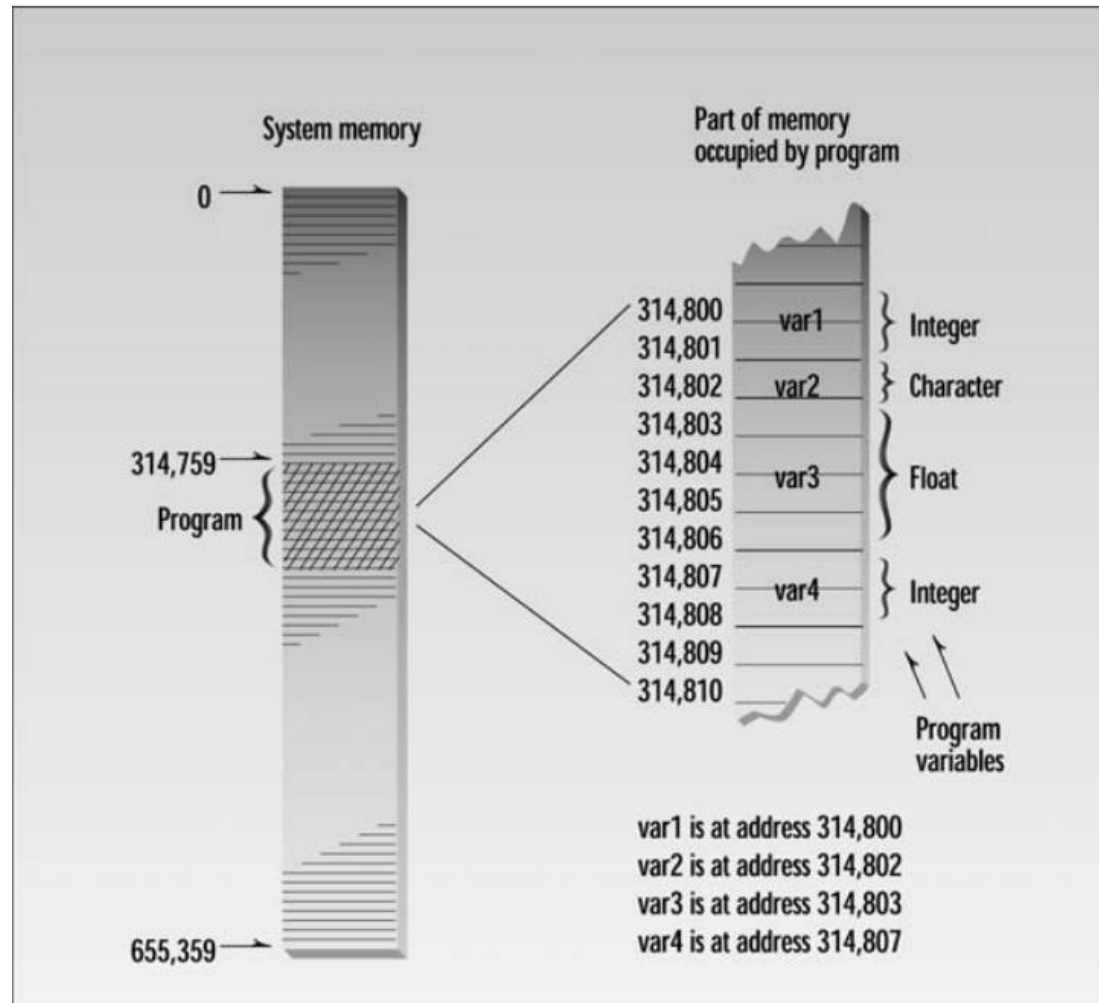
Variable name: `var`

Variable value:

4

Variable address: `0x006ae3`

Pointers – memory map



Pointers – print memory address ‘&’

```
int main()
{
    int i = 4, j = 5, k = 6;
    std::cout << i << ", " << j << ", " << k << '\n';

    // following prints 'address of' (aka &) of the variables
    std::cout << &i << ", " << &j << ", " << &k << '\n';
    return 0;
}
```

**Print the address of the
variables**

- Memory addresses aren't the same when you rerun the program
- Memory addresses aren't the same across different machines

Declaring pointers

- Great! We can print addresses of the variable
- How can store the address of the variable in another variable?
- `int* ptrI;`
 - Read the statement from right to left – “we have defined a variable which is a pointer to an int”.

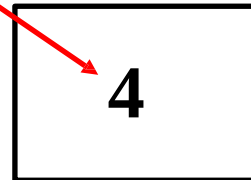
```
int main()
{
    int i = 4;
    int j = i;
    int* ptrI = &i;
    std::cout << ptrI << ", " << &i;

    return 0;
}
```

**j and i has the
same value
but not the
same address**

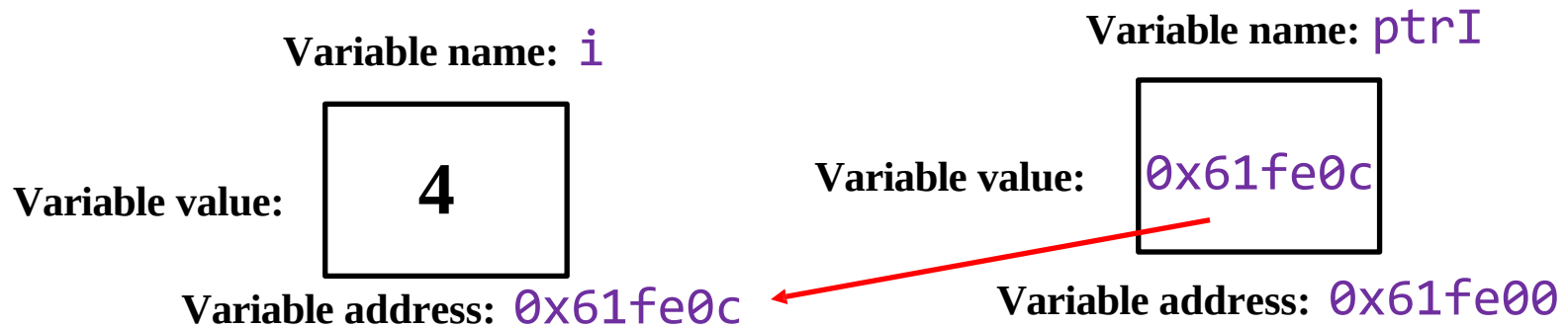
Variable name: `i`

Variable value:



Variable address: `0x61fe0c`

**ptrI points to the
memory address of i**



```
int i = 4;
```

```
int* ptrI = &i;
```

Pointers

- Pointers point to a memory address **AND** they also have their own memory address.
- Whats's the output of the code?

i=4, j=4

&i=0x61fe1c, &j=0x61fe18 ptrI=0x61fe1c

&ptrI=0x61fe10

```
int main(int argc, char ** argv)
{
    int i = 4;
    int j = i;
    int* ptrI = &i;

    std::cout << "i=" << i << ", j=" << j << '\n';
    std::cout << "&i=" << &i << ", &j=" << &j << '\n';
    std::cout << "ptrI=" << ptrI << '\n';

    // ptrI has its own memory address
    std::cout << "&ptrI=" << &ptrI << '\n';

    return 0;
}
```



Taking stock!

- Two variables can have same value but different memory addresses
- Pointers points to memory addresses
- But pointers have their own memory address! They are a variable in its own right

Pointers - Initialisation

- How can we initialise a pointer if we don't know the value upfront

- Initialisation 1:

```
int* ptrI = nullptr;
```

- Initialisation 2:

```
int* ptrI = &i;
```

Dereference operator

- How can we access value stored at a particular address?

```
✓ int main()
{
    int i = 4;
    int* ptrI = &i;
    std::cout << ptrI << ", " << *ptrI << "\n" ;

    return 0;
}
```

Dereference operator (gives you the value stored at the memory address)

- NOTE:** make sure you understand the dual purpose of '*' here.
 - * with type is declaring a pointer
 - * with variable name dereferences the pointer

What should be the output?

```
int main()
{
    int i = 4;
    int* ptrI = &i;

    *ptrI = 44;
    std::cout << ptrI << ", " << *ptrI << ", " << i << "\n" ;

    return 0;
}
```

- We modify the value of 'i' indirectly using the pointer!!!
-

Pointer types

- Sample code shows how we can declare other pointer types

```
int main(int argc, char ** argv)
{
    int i    = 4;
    float f  = 5.F;
    size_t s = 1u;
    double d = 1.;
    char c   = 'c';

    int*   pi  = &i;
    float* pf  = &f;
    size_t* ps = &s;
    double* pd = &d;
    char*   pc = &c;

    int*   ppi  = nullptr;
    float* ppf  = nullptr;
    size_t* pps = nullptr;
    double* ppd = nullptr;
    char*   ppc = nullptr;

    return 0;
}
```

Optional – `void*` and `reinterpret_cast`



- We can declare a general purpose pointer – `void*`

```
✓ int main(int argc, char ** argv)
{
    int i = 10;
    double d = 200;
    int* ppi = &i;
    double* ppf = &d;
    // ppf = ppi; // can't assign int* to double*

    void* pv = nullptr;
    ✓ pv = ppf; // CAN assign float* to void*
    // ppf = pv; // can't assign void* to double*

    return 0;
}
```

- Casting between pointers types require `reinterpret_cast`. DON'T USE IT. OR USE IT WITH CARE.



Pointer Arithmetic



Pointer Arithmetic

Let's say you have a pointer pointing to an integer.

Note that this is an **int** type pointer.

ptr → **100** (Note this is not the address where **ptr** is stored).

Adding 1 to it, will cause your pointer to increment by **1*sizeof(int)**.

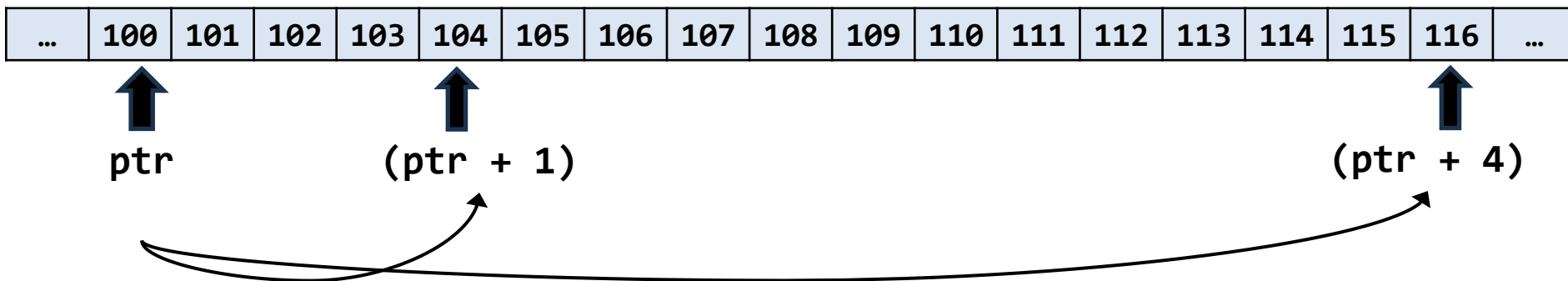
(ptr + 1) → 100 + (1*4) → 104

(ptr + 4) → 100 + (4*4) → 116

Note

Using base10 system
to represent
addresses.
Keep things easy to
understand.

*This is a snippet of
some of the memory
addresses.*





Pointer Arithmetic

Subtraction behaves in a similar way.

Assuming that, again, `ptr` $\rightarrow$ 100.

`(ptr - 1)` $\rightarrow$ 100 - (1*4) $\rightarrow$ 96

`(ptr - 4)` $\rightarrow$ 100 - (4*4) $\rightarrow$ 84

What if pointer was pointing at a **double**?

`ptr = 1000.`

`(ptr + 3)` $\rightarrow$???

Pointer Arithmetic



Question

What will happen if I dereference any of these pointers (lines 9 through 11)?

```
1  #include <iostream>
2
3  int main()
4  {
5      int value{ 7 };
6      int* ptr{ &value };
7
8      std::cout << ptr << '\n';
9      std::cout << ptr+1 << '\n';
10     std::cout << ptr+2 << '\n';
11     std::cout << ptr+3 << '\n';
12
13     return 0;
14 }
```

```
PS C:\Users\mohammad.salman\OneDrive -
1 2023\Lectures\Week 3\codes> .\a.exe
0x61fe14
0x61fe18
0x61fe1c
0x61fe20
```

Arrays in Memory

Let's say we have an array,

```
int prime[5]{ 2, 3, 5, 7, 11 };
```

Its representation will be as follows.

0	1	2	3	4
2	3	5	7	11
0x10	0x14	0x18	0x22	0x26

Recall from DSA that in an array, all elements are **contiguous**, i.e. the elements are next to one another.

Note that every entity **has to be saved in memory**.
In this case, we have **five integers (4 bytes each)**.

Array Structure



So basically,

```
&prime[0] = start_address + 0*sizeof(dtype),  
&prime[1] = start_address + 1*sizeof(dtype),  
&prime[2] = start_address + 2*sizeof(dtype),  
&prime[3] = start_address + 3*sizeof(dtype),  
And so on...
```



Using Pointer Arithmetic to Traverse an Array

Putting pointer arithmetic and arrays together...

```
#include <iostream>
```

```
int main()
```

```
{
```

```
    int array[]{ 9, 7, 5, 3, 1 };
```

```
    std::cout << &array[1] << '\n';
```

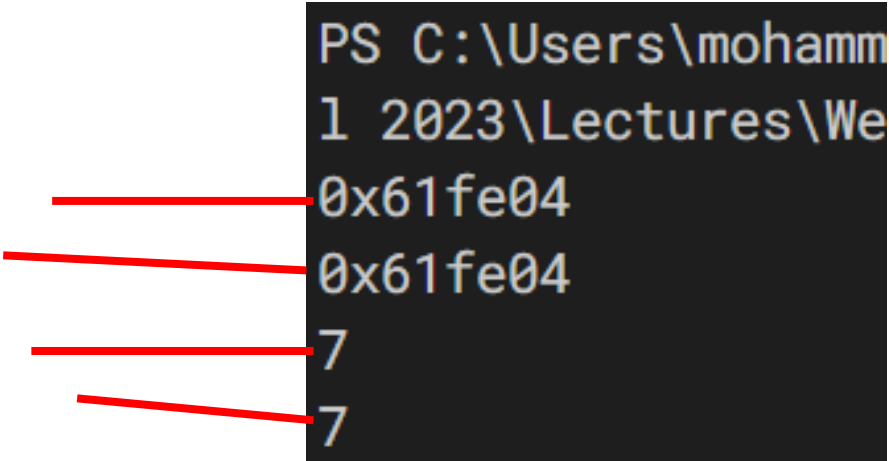
```
    std::cout << array+1 << '\n';
```

```
    std::cout << array[1] << '\n';
```

```
    std::cout << *(array+1) << '\n';
```

```
    return 0;
```

```
}
```



```
PS C:\Users\mohamm  
1 2023\Lectures\We  
0x61fe04  
0x61fe04  
7  
7
```

Traversing an array using the concepts that we've studied thus far.

Is this initialization fine or should be something like the following?

`int* ptr{ &array }; ?`

```
34 #include<iostream>
35
36 int main()
37 {
38     int array[] { 9, 7, 5, 3, 1 };
39
40     int len { sizeof(array) / sizeof(array[0]) };
41
42     // Alternative 1
43     for (int i{0}; i < len; i++)
44     {
45         std::cout << *(array + i) << ' ';
46     }
47     std::cout << '\n';
48
49
50     // Alternative 2
51     for (int* ptr{ array }; ptr < (array + len); ptr++)
52     {
53         std::cout << *ptr << ' ';
54     }
55     std::cout << '\n';
56
57
58     /* NOTE
59      - Generally, you don't need to use pointers to loop this way.
60      - Array notation (indexing) is easier to use, and less error-prone.
61      - However, this demonstrates the concepts that we have talked about quite well.
62      */
63
64     return 0;
65 }
66
67
68
69
```

++ operator : Arrays and pointers

- Spot the difference!
- What should be the output of the 2 snippets?

```
✓ int main(int argc, char ** argv)
{
    double darr[3] = {10., 7., 17.1};
    std::cout << darr << '\n';

    ✓ for(int i = 0; i < 3; ++i)
    {
        cout << *(darr + i) << '\n';
    }

    return 0;
}
```

```
✓ int main(int argc, char ** argv)
{
    double darr[3] = {10., 7., 17.1};
    std::cout << darr << '\n';

    ✓ for(int i = 0; i < 3; ++i)
    {
        cout << *(darr++) << '\n';
    }

    return 0;
}
```

- Array address is const. You can't modify it. We may see more constness of pointers later on in the course

++ operator : Arrays and pointers

- What did we do?
- Assigned array address to pointer variable
- We can increment that pointer variable that it not **const**

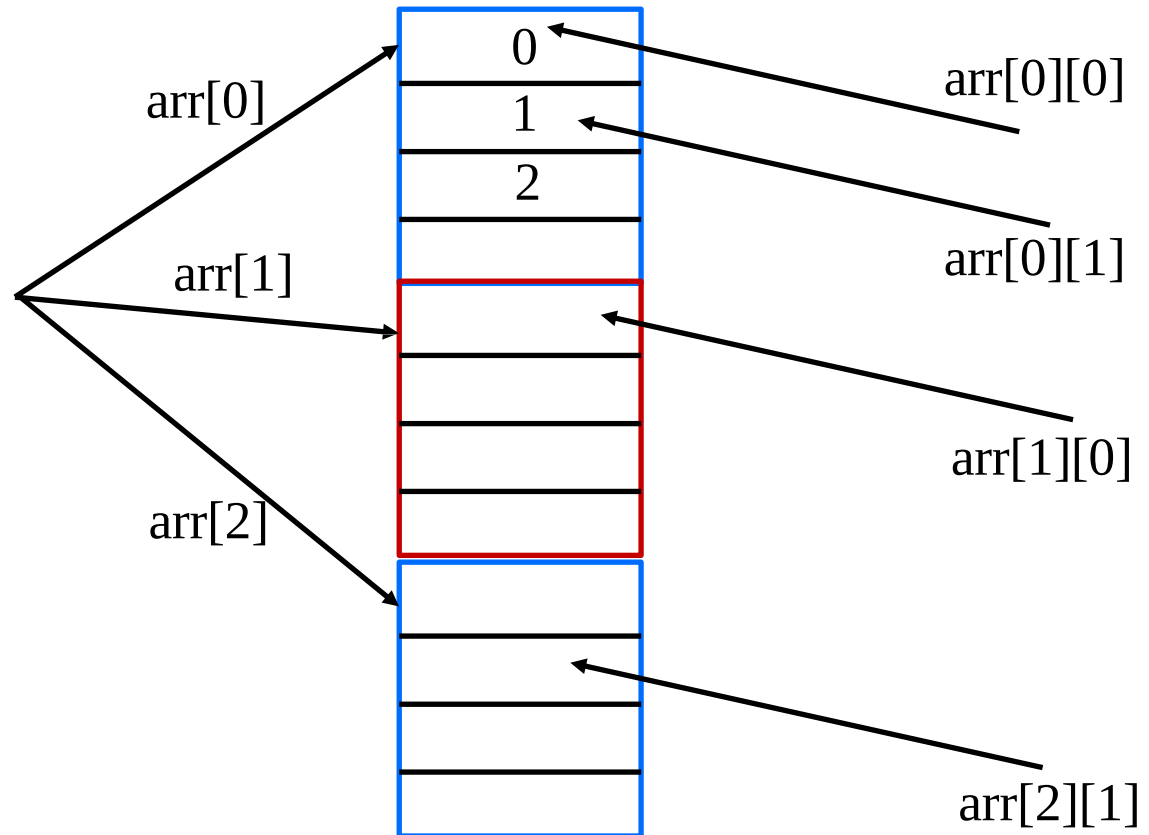
```
int main(int argc, char ** argv)
{
    double darr[3] = {10., 7., 17.1};
    double* pd = darr;

    for(int i = 0; i < 3; ++i)
    {
        cout << *(pd++) << '\n';
    }

    return 0;
}
```

2D arrays and pointers

```
int main(int argc, char ** argv)
{
    int arr[3][4] = {0, 1, 2};
    std::cout << arr[0] << '\n';
    std::cout << arr[1] << '\n';
    std::cout << arr[2] << '\n';
    return 0;
}
```



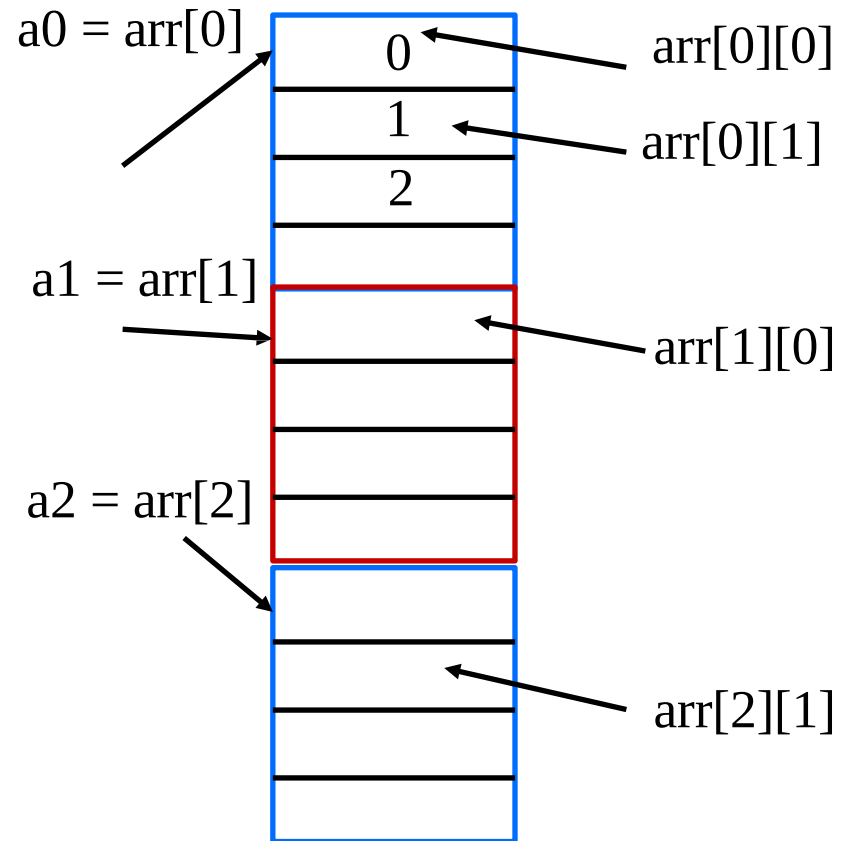
2D arrays and pointers



```
int main(int argc, char ** argv)
{
    int arr[3][4] = {0, 1, 2};

    int* a0 = arr[0];
    int* a1 = arr[1];
    int* a2 = arr[2];

    std::cout << arr[0] << ", " << a0 << '\n';
    std::cout << arr[1] << ", " << a1 << '\n';
    std::cout << arr[2] << ", " << a2 << '\n';
    return 0;
}
```



Homework: What's the output?

- Is there anything strange?

```
int main(int argc, char ** argv)
{
    int arr[3] = {0, 1, 2};
    char carr[20] = {'h','e','l','l','o', ' ', 'w', 'o', 'r', 'l', 'd'};
    std::cout << arr << '\n';
    std::cout << carr << '\n';
    return 0;
}
```



Size of a pointer

- Dependent on the system.
 - For 64-bit systems, size would be 8 bytes (64 bits)
 - For 32-bit systems, size would be 4 bytes (32 bits)
- For our purposes, assume that a pointer will be 8 bytes, unless otherwise specified in the question.
- Note that it doesn't matter how big the actual entity is to which the pointer is pointing to.
- It also doesn't matter what the data type of that entity is.
- The amount of space required by an address (and hence a pointer) is the same.

As such, the space required by all types of pointers is the same (4 or 8 bytes).
